# Homework 1

# Reconfigurable MAC unit

Course's GitHub repository: https://github.com/VVIPLab/ece284fa26

**Task:**

Please complete the *mac.v* file located in the *"hardware/w1/hw1/verilog/"* directory of the GitHub repository. **Do not make any changes to the other files (including the testbench).**

Testbench, data files, and template codes for the MAC unit are already provided in the repository.

Module Interface & Registers:

- **Inputs:** *A* (signed 8-bit), *B* (signed 8-bit), *format* (1-bit), *acc* (1-bit), *clk* (1-bit), *reset* (1-bit).
- **Outputs:** *out* (signed 16-bit)

Can refer to this link to understand more about arithmetic operations in Verilog:
https://chipverify.com/verilog/verilog-arithmetic-operators

MAC module Behavior:

- At every rising edge of *clk*, the two inputs (*A* and *B*) are captured into registers *a_q* and *b_q*, respectively.
- If *reset* == 1, internal register "*psum_q*" becomes zero (Reset is synchronous).
- If *acc* == 1, *psum_q* is updated with *psum_q + (a_q * b_q)* at the next rising edge.
- If *acc* == 0, *psum_q* is updated with *(a_q * b_q)* at the next rising edge. The previous value of *psum_q* is not accumulated.
- If *format* == 0, the MAC is performed in a signed 2's complement number system.
- If *format* == 1, the MAC operation is performed in a sign and magnitude system.
- The *out* signal must follow the number representation specified by "*format*".

General information:

- Assume that overflow does not occur.

- Hardware efficiency does not matter.
- You can create more intermediate nets/wire/reg if needed
- You can assume that "*format*" remains stable throughout normal operation and will only change after a reset. Consequently, mid-flight transitions (e.g., *format*=0 with non-zero *psum_q* followed by *format*=1) are not considered and need not be supported.

**Deliverables:**

1) Code: Upload your completed mac.v file.
2) Submit a report containing the following:
   - Five waveforms. Each must clearly display the reset, clock, acc, format, two inputs (A and B), and the out signals:
     - One complete overall waveform
     - Four zoomed-in parts of the waveform showcasing the four combinations of format (0,1) and acc (0,1). Ensure that the inputs and output values are clearly legible in each snippet
   - Code snippet of your mac.v file
   - Include the path of your working directory in the UCSD Linux Cloud

**Common Q&As:**

1. Which specific Linux Cloud server ought to be selected?

   It is advised to log into an ECE ieng6 Server Remote Desktop instance to access the prepared environment. (Alternative CentOS or Linux Mint remote desktops might lack the pre-configured parameters, so verify accordingly.)

2. What is the workflow on the Linux Cloud?

   If you want to work on the UCSD Linux Cloud, you can follow these steps:

   a. Launch the terminal and enter "*ECE284_FA26_001*". This navigates automatically to /home/linux/ieng6/ECE284_FA26_001/xxxxxx (where xxxxxx represents your UCSD username) while setting up the course environment. (Because of the instructor account setup noted during lecture, shortcuts might be missing initially; configure them manually by running the following lines:
   *alias iveri="iverilog -o compiled -c"*

   *alias irun="vvp compiled"*

   *alias wave="gtkwave"*)

b. Within your current folder, execute a git clone for the repository:

   *git clone https://github.com/VVIPLab/ece284fa26.git*

c. Run *cd ece284fa26/hardware/wk1/hw1/verilog* and edit mac.v utilizing a text editor like vim or nano.
d. Run cd .. to return to ece284fa26/hardware/wk1/hw1, then run *iveri filelist* to compile your design files.
e. Run *irun* to launch the compiled binary.
f. Run *gtkwave mac_tb.vcd* to inspect simulation output waveforms.

-